



MSPR TPRES01 - DEVIA FS

Développement et Déploiement d'une Application d'Analyse des Pandémies

Certification Développeur en Intelligence Artificielle et Data Science (RNCP 36581)

Projet réalisé dans le cadre du Bloc de Compétences 1 :

“Créer un modèle de données d'une solution I.A en utilisant des méthodes de Data Science”

Établissement : EPSI Lyon

Année académique : 2024-2025

Auteurs :

- Iliana Benchikh (Développeuse Back-End)
- Brahim (Développeur Front-End / Back-End)
- Anas (Développeur Back-End)
- Akram (Développeur Front-End)

Table des matières

I) Introduction.....	4
1) Contexte et enjeux du projet	4
2) Objectif du projet	4
3) Données utilisées	5
4) Portée et livrables du projet.....	5
II) Architecture du projet.....	5
III) Technologies utilisées.....	8
1) Langage de programmation	8
2) Environnement de développement	10
3) Traitement et manipulation des données	5
4) Stockage et gestion des données	13
5) Développement de l'API REST	14
6) Visualisation des données	15
IV) Collecte des données	15
1) Contenu du dataset	16
V) Pipeline ETL (Extract, Transform, Load).....	17
1) Vérification de la connexion à PostgreSQL.....	17
2) Extraction des données	18
3) Nettoyage et transformation des données	19
a) Chargement et normalisation des données	19
b) Correction des incohérences et ajout de colonnes dérivées	20
c) Correction des valeurs négatives et incohérentes	20
d) Suppression des valeurs aberrantes.....	21
e) Ajout des indicateurs mortality_rate et recovery_rate	21
f) Suppression des lignes avec 0 cas	22
g) Sauvegarde des données nettoyées	22
h) Insertion et mise à jour des données	23
Conclusion	24
VI) Entraînement des Données Pandémiques	25
1. Introduction.....	25
2. Structure Générale	25
Le script suit plusieurs étapes principales :.....	25
3. Extraction des Données.....	25
(a) Connexion à la Base PostgreSQL	26
(b) Récupération des Données.....	26

4. Prétraitement des Données	26
4.1 Filtrage des Données par Pays.....	26
5. Entraînement du Modèle	27
9. Conclusion	28
VII) API de consultation des données	28
1) Architecture de l'API	28
2) Connexion à la base de données.....	29
3) Modèle de données (Pydantic)	29
4) Endpoints de l'API Pandemics.....	31
a) Test de connexion à la base de données PostgreSQL.....	31
b) Récupération des données (GET)	32
c) Ajout de nouvelles données (POST)	33
d) Mise à jour des données (PUT)	36
e) Suppression d'une entrée (DELETE)	38
f) Documentation automatique avec Swagger	39
5) Endpoints de l'API Users	39
a) Création d'un utilisateur (POST /users/)	39
b) Récupération d'un utilisateur (GET /users/{user_id})	40
c) Mise à jour d'un utilisateur (PUT /users/{user_id})	40
d) Suppression d'un utilisateur (DELETE /users/{user_id})	41
VIII) Visualisation et exploration des données avec Power BI	42
1) Objectif de la visualisation des données	42
2) Connexion des données dans Power BI	42
3) Structuration des données	43
4) Tableaux de bord et visualisations	43
a) Sélection des filtres interactifs	43
b) Moyenne du nombre de décès et de guérisons par pays	44
c) Courbe d'évolution des cas, décès et guérisons.....	45
d) Moyenne des cas par région OMS.....	46
e) Répartition géographique des cas par pays	47
f) Indicateurs clés calculés	48
g) Interaction et expérience utilisateur	49
IX)..... Frontend	39
X) Benchmark des solutions	59

I) Introduction

1) Contexte et enjeux du projet

L'évolution rapide des pandémies mondiales, comme la COVID-19 et plus récemment la variole du singe (MPOX), a mis en lumière la nécessité de disposer de systèmes d'information performants pour analyser et anticiper ces crises sanitaires. En réponse à ces défis, une nouvelle division de recherche a été créée au sein de l'Organisation Mondiale de la Santé (OMS) afin de centraliser, nettoyer, analyser et visualiser les données historiques des pandémies.

Dans ce cadre, le projet **MSPR - DEVIA FS** s'inscrit dans une démarche de **développement d'une solution de traitement et de visualisation des données**, permettant aux chercheurs et décideurs d'accéder à des indicateurs clés et de modéliser des tendances épidémiologiques. L'objectif est de rendre les données accessibles et exploitables, tout en assurant leur qualité et leur pertinence.

2) Objectif du projet

Le projet vise à concevoir et déployer une solution **ETL (Extract, Transform, Load) couplée à une plateforme de datavisualisation** permettant d'exploiter efficacement les données de pandémie. Ce système permettra :

- **L'ingestion de données** provenant de sources ouvertes et fiables.
- **Le nettoyage et la structuration** des données pour garantir leur intégrité et leur cohérence.
- **Le stockage optimisé** des informations dans une base de données relationnelle.
- **La mise en place d'une API** pour l'accès aux données par les développeurs.
- **La création de tableaux de bord interactifs** facilitant l'analyse et l'interprétation des tendances.

3) Données utilisées

Pour ce projet, nous avons opté pour un jeu de données unique, issu de Kaggle, qui retrace **l'évolution de la pandémie de COVID-19** à travers différentes dimensions. Le dataset sélectionné contient des informations cruciales sur la propagation du virus, sa dynamique et son impact global.

Le fichier CSV exploité, nommé "covid_19_clean_complete.csv", contient des données issues de sources officielles telles que l'OMS, les CDC (Centers for Disease Control and Prevention) et d'autres organismes de santé publique. Il couvre une large période, englobant l'évolution de la pandémie de janvier 2020 à fin 2021, avec des mises à jour régulières.

Ce dataset fournit une granularité jour par jour, permettant une analyse fine de l'évolution du virus à travers 210 pays et territoires.

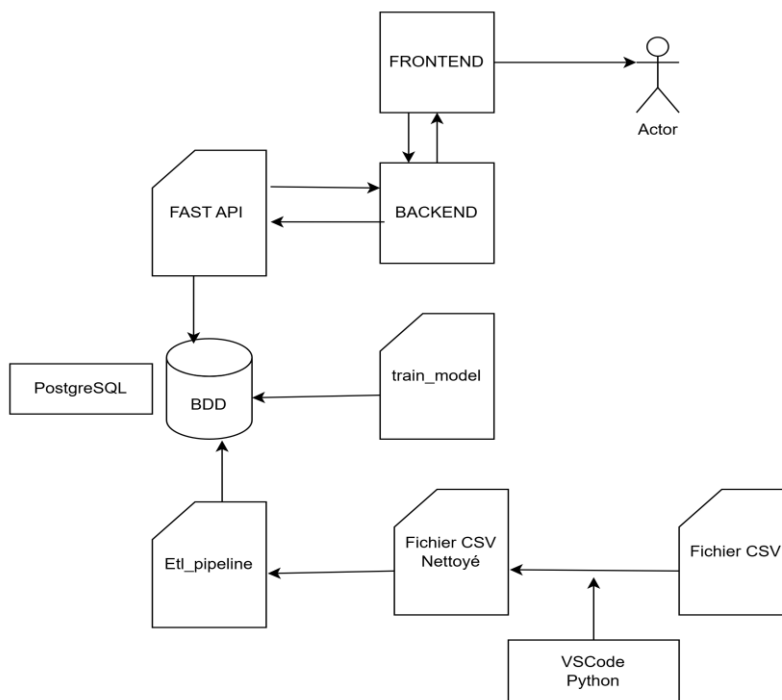
4) Portée et livrables du projet

Notre équipe de développement, mandatée par **l'OMS et le groupe ANALYZE IT**, doit fournir un ensemble de livrables garantissant une solution exploitable et évolutive :

1. **Un modèle de données** décrivant l'architecture des informations stockées.
2. **Une base de données relationnelle** optimisée pour le stockage et l'interrogation des données.
3. **Un pipeline ETL** pour l'extraction, le nettoyage et l'insertion des données dans la base.
4. **Une API RESTful** permettant d'interagir avec les données.
5. **Un tableau de bord interactif** pour la visualisation des indicateurs et tendances.
6. **Une documentation technique** décrivant les choix architecturaux, les outils utilisés et les processus mis en place.

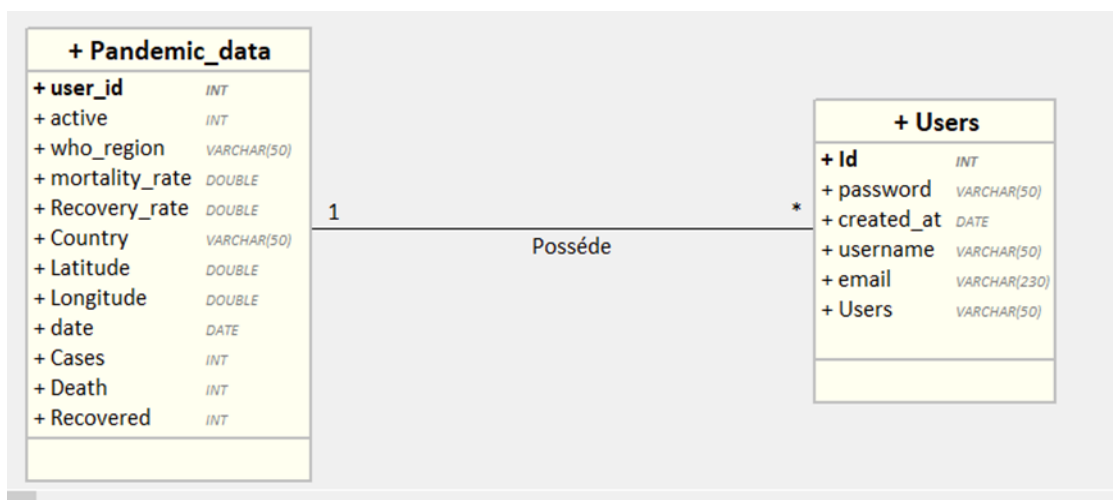
II) Architecture du projet

Diagramme de flux des données :



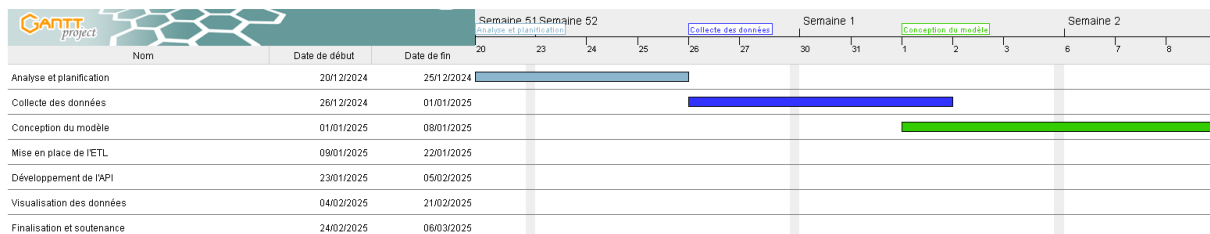
On a utilisé pour ce diagramme est **draw.io**.

UML :



On a utilisé pour cet UML est **Looping**.

Diagramme de gantt :



Nom	Date de début	Date de fin	Ressources
Analyse et planification	20/12/2024	25/12/2024	
Collecte des données	26/12/2024	01/01/2025	
Conception du modèle	01/01/2025	08/01/2025	
Mise en place de l'ETL	09/01/2025	22/01/2025	
Développement de l'API	23/01/2025	05/02/2025	
Visualisation des données	04/02/2025	21/02/2025	
Finalisation et soutenance	24/02/2025	06/03/2025	

On a utilisé pour ce projet gantt est **GantProject**.

III) Technologies utilisées

1) Langage de programmation

Backend :

Python



Python a été choisi comme langage principal en raison de sa popularité et de sa richesse en bibliothèques pour la data science et le développement web.

Pourquoi Python ?

- Écosystème riche pour le traitement des données (Pandas, NumPy, Scikit-learn).
- Facilité d'automatisation des tâches ETL avec Prefect.
- Frameworks web performants comme FastAPI et Flask.
- Interopérabilité avec PostgreSQL et Power BI.

Utilisation dans le projet :

- Manipulation des données avec Pandas.
- Automatisation du pipeline ETL avec Prefect.
- Développement de l'API REST avec FastAPI.
- Visualisation interactive des données avec Power BI.

Frontend :

HTML, CSS et JavaScript



HTML, CSS et JavaScript ont été choisis pour la création d'interfaces web modernes, responsives et dynamiques, permettant une interaction fluide avec l'utilisateur et l'API backend.

Pourquoi HTML, CSS et JavaScript ?

- **HTML** : Fournit la structure sémantique du contenu pour une bonne accessibilité.
- **CSS** : Permet de styliser et de mettre en forme les pages, d'appliquer des animations et d'assurer un design moderne et responsive.
- **JavaScript** : Apporte l'interactivité et la logique front-end, permettant de charger dynamiquement des composants et d'effectuer des requêtes asynchrones.

Utilisation dans le projet :

- Construction des pages de connexion, dashboards et formulaires en HTML.
- Application d'un design cohérent et d'animations via CSS (main.css, index.css, etc.).
- Logique front-end (CRUD, filtrage, pagination, stockage du token dans le localStorage) via des scripts JavaScript (index.js, pandemics.js, users.js).

HTTP-SERVER



Http-server est un serveur statique léger permettant de servir les fichiers HTML, CSS et JavaScript localement.

Pourquoi http-server ?

- Configuration minimale pour lancer un serveur web local.
- Rapide et simple à installer (une commande suffit).
- Permet de tester l'application front-end sans nécessiter un serveur plus complexe.

Utilisation dans le projet :

- Lancement du dossier frontend/public sur le port 8000 pour visualiser les pages.
- Test de l'intégration front-end / API en local (<http://127.0.0.1:8000>).
- Débogage et vérification de la cohérence de l'interface avant le déploiement.

2) Environnement de développement

Visual Studio Code (VS Code)



L'IDE utilisé pour ce projet est Visual Studio Code, un environnement de développement léger, rapide et personnalisable.

Pourquoi VS Code ?

- Excellente prise en charge de Python avec des extensions dédiées.
- Extensions puissantes pour le debug, l'exécution et l'analyse du code.
- Support de Jupyter Notebooks pour le prototypage et l'analyse exploratoire.

Utilisation dans le projet :

- Développement et exécution des scripts ETL.
- Debug et test des requêtes SQL dans PostgreSQL.
- Lancement et test de l'API FastAPI.

Git et GitHub



Git et GitHub sont utilisés pour la collaboration en équipe et le suivi de l'historique du code.

Pourquoi Git et GitHub ?

- Contrôle de version distribué permettant un travail simultané sans conflits.

- Plateforme centralisée pour les merge requests, la revue de code et la documentation.
- Possibilité de mettre en place des branches protégées et des commits conventionnels.

Utilisation dans le projet :

- Création de branches dédiées pour chaque fonctionnalité ou correctif.
- Merge requests pour intégrer le code dans la branche principale après revue.
- Adoption de conventional commits pour assurer la lisibilité et la traçabilité des modifications.

3) Design et Prototypage

Figma



Figma est utilisé pour la conception des maquettes et la validation du design de l'interface utilisateur.

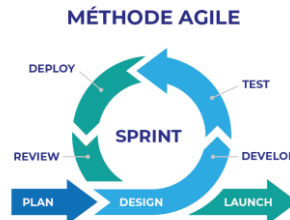
Pourquoi Figma ?

- Outil de design collaboratif en ligne, permettant de travailler simultanément.
- Possibilité de créer des prototypes interactifs pour valider rapidement les choix UX/UI.
- Facilité de partage et de commentaire avec l'équipe de développement.

Utilisation dans le projet :

- Création des maquettes pour la page de connexion, les dashboards, etc.
- Validation du design (typo, couleurs, agencement) avant l'implémentation CSS.
- Référence visuelle continue pour maintenir une cohérence graphique.

Méthodologie Agile et Gestion de Projet



La méthodologie Agile et une gestion de projet rigoureuse ont été adoptées pour favoriser la flexibilité et la collaboration au sein de l'équipe.

Pourquoi la méthodologie Agile et une gestion de projet structurée ?

- Itérations courtes pour une livraison continue de fonctionnalités.
- Collaboration accrue via des stand-ups quotidiens et des rétrospectives.
- Capacité à réagir rapidement aux retours et aux changements de priorités.
- Traçabilité et organisation via des outils de planification.

Utilisation dans le projet :

- Découpage du développement en sprints avec des objectifs précis.
- Suivi des tâches et priorisation.
- Revue de code systématique via des merge requests sur GitHub.
- Adoption de branches protégées et de commits conventionnels pour garantir la qualité et la cohérence du code.

4) Traitement et manipulation des données

Pandas



Pandas est l'outil clé du projet pour la manipulation des données.

Pourquoi Pandas ?

- Manipulation efficace des DataFrames.
- Nettoyage et transformation des données avec des opérations vectorisées.
- Gestion des fichiers CSV avec une prise en charge avancée du parsing.

Utilisation dans le projet :

- Chargement et nettoyage des données depuis le fichier Kaggle.
- Ajout de colonnes calculées comme le taux de mortalité.
- Filtrage et correction des valeurs aberrantes.

Prefect

PREFECT

Prefect est utilisé pour l'automatisation du pipeline ETL.

Pourquoi Prefect ?

- Plus simple à configurer qu'Apache Airflow.
- Compatible avec du stockage local et cloud.
- Gestion avancée des erreurs et logs.

Utilisation dans le projet :

- Orchestration du pipeline ETL (Extraction, Transformation, Chargement).
- Surveillance et logs des processus ETL.
- Planification et exécution automatisée des tâches.

5) Stockage et gestion des données

PostgreSQL



La base de données choisie est PostgreSQL, une solution open-source robuste et performante.

Pourquoi PostgreSQL ?

- Support natif des types JSON et des requêtes avancées.
- Fiabilité accrue pour les transactions complexes.
- Optimisation possible avec des index sur les colonnes clés.

Utilisation dans le projet :

- Stockage des **données nettoyées**.
- Indexation sur country et date pour **accélérer les requêtes**.
- Connexion avec **Power BI** pour la visualisation des données.

6) Développement de l'API REST

FastAPI



FastAPI a été sélectionné pour le développement de l'API.

Pourquoi FastAPI ?

- Beaucoup plus rapide que Flask grâce à l'asynchronisme.
- Documentation automatique Swagger intégrée.
- Validation des entrées avec Pydantic pour éviter les erreurs.

Utilisation dans le projet :

- Exposition des données COVID-19 via une API REST.
- Création d'endpoints GET, POST, PUT, DELETE.

- Sécurisation des entrées et validation des types de données.

7) Visualisation des données

Power BI



Power BI a été choisi pour l'analyse interactive des données.

Pourquoi Power BI ?

- Facile à prendre en main pour créer des rapports interactifs.
- Connexion directe avec une API.
- Filtres interactifs pour une exploration des données en profondeur.

Utilisation dans le projet :

- Suivi en temps réel des tendances COVID-19.
- Exploration par pays et région OMS.
- Comparaison des taux de mortalité et de récupération.

IV) Collecte des données

Pour assurer une analyse fiable et précise de l'évolution de la pandémie du COVID-19, nous avons choisi d'utiliser un dataset issu de Kaggle, intitulé "covid_19_clean_complete.csv". Ce dataset a été sélectionné sur la base des critères suivants :

- **Données officielles** provenant de sources reconnues comme l'**OMS** et le **CDC**.
- **Granularité quotidienne** permettant une analyse fine des tendances pandémiques.
- **Couverture géographique étendue** couvrant **210 pays et territoires**.
- **Informations complètes** sur les **cas confirmés**, les **décès**, les **guérisons**, les **cas actifs** et la **répartition par région OMS**.

Ce dataset nous permet de **comprendre l'évolution du virus sur une période donnée**, d'explorer les **tendances épidémiologiques**, et d'établir des **indicateurs pertinents** pour la visualisation et l'analyse.

1) Contenu du dataset

Le fichier CSV utilisé contient les colonnes suivantes :

Colonne	Description
Province/State	Nom de la province ou de l'État (si applicable).
Country/Region	Pays ou région géographique où les données sont collectées.
Lat	Latitude de la localisation.
Long	Longitude de la localisation.
Date	Date d'enregistrement des données (granularité quotidienne).
Confirmed	Nombre total de cas confirmés à cette date.
Deaths	Nombre total de décès recensés.
Recovered	Nombre total de patients guéris.
Active	Nombre total de cas actifs (Confirmed - Deaths - Recovered).
WHO Region	Région géographique selon la classification de l'OMS.

Ce fichier servira de source principale pour notre pipeline ETL.

V) Pipeline ETL (Extract, Transform, Load)

Un pipeline ETL (Extract, Transform, Load) a été mis en place afin d'assurer le traitement et l'intégration des données dans une base PostgreSQL. Ce pipeline est conçu avec Prefect, un outil d'automatisation et d'orchestration de flux de données.

1) Vérification de la connexion à PostgreSQL

Avant de lancer l'ETL, nous devons nous assurer que **PostgreSQL est accessible**. Un test de connexion est donc effectué **avant le chargement des données**. Ce test est réalisé en effectuant une **tentative de connexion répétée** si le serveur n'est pas immédiatement disponible.

```
# 🚀 Connexion PostgreSQL
def check_postgres():
    print("🔍 Vérification de la connexion à PostgreSQL...")
    for _ in range(5):
        try:
            conn = psycopg2.connect(
                dbname="pandemics",
                user="postgres",
                password="admin",
                host="localhost",
                port=5432
            )
            conn.close()
            print("✅ Connexion PostgreSQL réussie !")
            return True
        except psycopg2.OperationalError:
            print("⚠️ PostgreSQL inaccessible, nouvelle tentative dans 5s...")
            time.sleep(5)
    print("❌ PostgreSQL inaccessible après plusieurs tentatives.")
    return False
```

Explication du code :

- Tentative de connexion jusqu'à 5 fois avec une pause de 5 secondes entre chaque essai.
- Gestion des erreurs OperationalError qui surviennent si PostgreSQL n'est pas encore prêt.

Affichage d'un message d'erreur clair si le serveur est indisponible après plusieurs tentatives.

2) Extraction des données

La première étape du pipeline consiste à **extraire** les données depuis le dossier local.

Le script suivant permet d'automatiser le téléchargement et l'extraction des fichiers de données :

```
# 🚩 Dossier de stockage
BASE_DIR = os.path.dirname(os.path.abspath(__file__)) # Récupère le chemin absolu du script
DATA_FOLDER = os.path.join(BASE_DIR, "data")
CSV_FILE = os.path.join(DATA_FOLDER, "covid_19_clean_complete.csv")

# 🚩 Vérifier et créer le dossier si nécessaire
if not os.path.exists(DATA_FOLDER):
    os.makedirs(DATA_FOLDER)
```

Explication du code :

- Création automatique d'un dossier data/ pour stocker les fichiers téléchargés.
- Gestion des erreurs en cas d'échec du téléchargement.

3) Nettoyage et transformation des données

Une fois les données extraites, elles doivent être **prétraitées et transformées** pour être exploitables.

a) Chargement et normalisation des données

Nous chargeons le fichier CSV et effectuons un renommage des colonnes pour faciliter l'analyse :

```
print(f"📁 Chargement et nettoyage du fichier {CSV_FILE}...")
df = pd.read_csv(file_path, encoding="utf-8", delimiter=";", on_bad_lines="skip")

df = df.rename(columns={
    "Country/Region": "country",
    "Lat": "latitude",
    "Long": "longitude",
    "Date": "date",
    "Confirmed": "cases",
    "Deaths": "deaths",
    "Recovered": "recovered",
    "Active": "active",
    "WHO Region": "who_region"
})
```

Explication du code :

- **Chargement des données brutes** en mémoire via **Pandas**.
- **Renommage des colonnes** pour standardiser les libellés et assurer une meilleure lisibilité.
- **Suppression des lignes problématiques** avec `on_bad_lines="skip"` pour éviter les erreurs lors du chargement.

b) Correction des incohérences et ajout de colonnes dérivées

Nous devons nous assurer que chaque colonne contient des valeurs de type cohérent :

```
df["date"] = pd.to_datetime(df["date"], errors='coerce')
df["cases"] = pd.to_numeric(df["cases"], errors='coerce').fillna(0).astype(int)
df["deaths"] = pd.to_numeric(df["deaths"], errors='coerce').fillna(0).astype(int)
df["recovered"] = pd.to_numeric(df["recovered"], errors='coerce').fillna(0).astype(int)
df["active"] = pd.to_numeric(df["active"], errors='coerce').fillna(0).astype(int)
```

Explication du code :

- **Conversion de la colonne date** en format datetime pour permettre des filtres temporels.
- **Transformation des valeurs en entiers (int)** pour cases, deaths, recovered, et active.
- **Gestion des valeurs NaN** en remplaçant les absences par 0.

c) Correction des valeurs négatives et incohérentes

Les colonnes recovered et active doivent être vérifiées pour éviter les valeurs erronées :

```
# Correction de 'recovered' si négatif et recalcul de 'active'
df.loc[df["recovered"] < 0, "recovered"] = df["cases"] - df["deaths"]
df["active"] = df["cases"] - df["deaths"] - df["recovered"]

# Correction des valeurs négatives de 'active'
df["active"] = df["active"].apply(lambda x: max(0, x))
```

Explication du code :

- Si **recovered** est négatif, il est recalculé comme cases - deaths.
- Recalcul des **active cases** pour refléter correctement la réalité.
- Suppression des **valeurs active négatives** en prenant $\max(0, x)$.

d) Suppression des valeurs aberrantes

Avant de sauvegarder les données, nous devons **éliminer les valeurs invalides** :

```
# Suppression des valeurs aberrantes
df = df.dropna(subset=["date"])
df = df[df["cases"] >= 0]
df = df[df["deaths"] >= 0]
df = df[df["recovered"] >= 0]
df = df[df["active"] >= 0]
```

Explication du code :

- **Supprime les lignes sans date** (NaN dans la colonne date).
- **Élimine toute valeur négative** pour éviter des erreurs d'interprétation.

e) Ajout des indicateurs mortality_rate et recovery_rate

Nous ajoutons deux nouvelles colonnes pour **analyser les tendances** :

```
# Ajout de nouvelles colonnes dérivées
df["mortality_rate"] = (df["deaths"] / df["cases"]).fillna(0).apply(lambda x: min(100, x * 100))
df["recovery_rate"] = (df["recovered"] / df["cases"]).fillna(0).apply(lambda x: min(100, x * 100))
```

Explication du code :

- Taux de mortalité = (deaths / cases) * 100, limité à 100%.
- Taux de récupération = (recovered / cases) * 100, limité à 100%.

f) Suppression des lignes avec 0 cas

Nous supprimons les enregistrements sans cas confirmés, qui n'apportent aucune valeur analytique.

```
# Suppression des lignes avec 0 cas pour éviter du bruit
df = df[df["cases"] > 0]
```

Explication du code :

- Seules les lignes avec cases > 0 sont conservées.
- Évite d'avoir des enregistrements inutiles dans la base.

g) Sauvegarde des données nettoyées

Enfin, nous sauvegardons les données **prêtes à être insérées dans PostgreSQL** :

```
cleaned_file = os.path.join(DATA_FOLDER, "covid_cleaned.csv")
df[["country", "latitude", "longitude", "date", "cases", "deaths", "recovered", "active", "who_region", "mortality_rate", "recovery_rate"]]
print(f"✅ Données nettoyées et sauvegardées dans {cleaned_file}")
return cleaned_file
```

Explication du code :

- Génération d'un fichier covid_cleaned.csv contenant les données propres et enrichies.
- Suppression des indices (index=False) pour un formatage plus propre.

h) Insertion et mise à jour des données

Nous créons la table `pandemic_data` si elle n'existe pas :

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS pandemic_data (
    country TEXT,
    latitude FLOAT,
    longitude FLOAT,
    date DATE,
    cases INT,
    deaths INT,
    recovered INT,
    active INT,
    who_region TEXT,
    mortality_rate FLOAT,
    recovery_rate FLOAT,
    PRIMARY KEY (country, date)
)
""")
```

Puis les données sont insérées dans la base tout en évitant les doublons :

```
df = pd.read_csv(cleaned_file)
for _, row in df.iterrows():
    cursor.execute("""
        INSERT INTO pandemic_data (country, latitude, longitude, date, cases, deaths, recovered, active, who_region, mortality_rate, recovery_rate)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        ON CONFLICT (country, date)
        DO UPDATE SET
            cases = EXCLUDED.cases,
            deaths = EXCLUDED.deaths,
            recovered = EXCLUDED.recovered,
            active = EXCLUDED.active,
            who_region = EXCLUDED.who_region,
            mortality_rate = EXCLUDED.mortality_rate,
            recovery_rate = EXCLUDED.recovery_rate;
    """, (row["country"], row["latitude"], row["longitude"], row["date"], row["cases"], row["deaths"], row["recovered"], row["active"], row["who_region"], row["mortality_rate"], row["recovery_rate"]))

conn.commit()
cursor.close()
conn.close()
print("✅ Données chargées dans PostgreSQL")
```

Explication du code :

- **Insertion optimisée avec PostgreSQL.**
- **Mise à jour automatique des enregistrements existants** en cas de conflit.

Conclusion

L'approche utilisée pour le traitement des données dans cette pipeline ETL repose sur plusieurs étapes critiques afin d'assurer l'intégrité, la qualité et l'accessibilité des données pour l'analyse des pandémies.

1. **Robustesse de la Connexion PostgreSQL** : La mise en place d'une vérification automatique de la connexion garantit une récupération fiable des données et empêche les interruptions lors du chargement des informations.
2. **Normalisation et Nettoyage des Données** :
 - a. **Élimination des valeurs aberrantes** : Les données sont filtrées pour supprimer les entrées avec des incohérences, telles que des cas négatifs ou des données incorrectement formatées.
 - b. **Recalcul des indicateurs clés** : Les champs `active`, `mortality_rate` et `recovery_rate` sont recalculés pour garantir une cohérence avec les valeurs de `cases`, `deaths` et `recovered`. Cette étape est essentielle pour éviter les erreurs de reporting et assurer une meilleure fiabilité des analyses.
 - c. **Conversion des types de données** : Les dates sont transformées au format `datetime` et les champs numériques sont convertis pour éviter toute perte d'information.
 - d. **Traitement des valeurs manquantes** : Les valeurs nulles dans les champs critiques sont remplacées ou recalculées pour éviter les erreurs lors des traitements analytiques.
3. **Prévention des Erreurs et Doublons** : L'insertion avec `ON CONFLICT` dans PostgreSQL permet de maintenir une cohérence dans les données sans générer de doublons tout en mettant à jour les informations existantes.
4. **Automatisation avec Prefect** : L'intégration de Prefect facilite l'exécution et la gestion du pipeline ETL en assurant une exécution séquentielle et en permettant une gestion des erreurs plus efficace.
5. **Stockage et Traçabilité** : Les données nettoyées sont stockées dans des fichiers intermédiaires (csv) afin de conserver une trace des modifications avant leur ingestion dans la base.

Cette approche modulaire et robuste permet de garantir une mise à jour fiable et continue des données, essentielle pour une analyse précise et pertinente des pandémies en temps

réel. En cas d'évolution des besoins, ce pipeline peut être facilement ajusté pour intégrer de nouvelles sources de données ou de nouveaux traitements spécifiques.

VI) Entraînement des Données Pandémiques

1. Introduction

Ce document détaille le processus d'extraction, de prétraitement et d'entraînement d'un modèle de régression linéaire pour prédire l'évolution des cas pandémiques par pays. Le modèle est entraîné sur les données extraites d'une base PostgreSQL et sauvegardé pour une utilisation ultérieure.

2. Structure Générale

Le script suit plusieurs étapes principales :

1. **Connexion à la base de données PostgreSQL.**
2. **Extraction des données** depuis la table `pandemic_data`.
3. **Prétraitement des données** (filtrage, gestion des valeurs manquantes, transformation des dates en jours écoulés).
4. **Séparation des données en ensembles d'entraînement et de test.**
5. **Entraînement d'un modèle de régression linéaire.**
6. **Évaluation du modèle** avec les métriques MAE et RMSE.
7. **Sauvegarde du modèle entraîné** au format `.pkl`.

3. Extraction des Données

(a) Connexion à la Base PostgreSQL

Le script utilise psycopg2 pour se connecter à la base de données et récupérer les données pandémiques.

```
# 🚀 Connexion PostgreSQL
def get_db_connection():
    return psycopg2.connect(
        dbname="pandemics",
        user="postgres",
        password="admin",
        host="localhost",
        port=5432
    )
```

(b) Récupération des Données

Les données sont extraites en filtrant uniquement les enregistrements où le nombre de cas est supérieur à 0 et sont triées par date.

```
# 🚀 Extraction des données
def fetch_pandemic_data():
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT date, country, cases, deaths, recovered, active, mortality_rate, recovery_rate
        FROM pandemic_data
        WHERE cases > 0
        ORDER BY date ASC;
    """)

    columns = ["date", "country", "cases", "deaths", "recovered", "active", "mortality_rate", "recovery_rate"]
    data = cursor.fetchall()

    cursor.close()
    conn.close()

    df = pd.DataFrame(data, columns=columns)
    df["date"] = pd.to_datetime(df["date"]) # Conversion en format datetime
    return df
```

4. Prétraitement des Données

4.1 Filtrage des Données par Pays

Les données sont filtrées pour ne conserver que celles correspondant au pays choisi pour l'entraînement du modèle.

```
# 🚀 Prétraitement des données
def preprocess_data(df, country):
    df = df[df["country"] == country].copy()

    # Transformation des dates en jours écoulés depuis la première observation
    df["days_since_start"] = (df["date"] - df["date"].min()).dt.days

    # Définition des features
    features = ["days_since_start", "deaths", "recovered", "active", "mortality_rate", "recovery_rate"]

    # Suppression des valeurs manquantes
    df = df.dropna(subset=features + ["cases"])

    # Définition des variables X (features) et y (cible)
    X = df[features]
    y = df["cases"]

    return X, y
```

5. Entraînement du Modèle

```
# 🚀 Entraînement du modèle et sauvegarde
def train_model(country="France"):
    df = fetch_pandemic_data()

    if df.empty or country not in df["country"].unique():
        raise ValueError(f"Aucune donnée disponible pour {country}")

    X, y = preprocess_data(df, country)

    # Séparation des données (80% entraînement, 20% test)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # Création et entraînement du modèle
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Évaluation
    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    # 🚀 Enregistrement du modèle dans le dossier `data`
    model_filename = os.path.join(DATA_FOLDER, f"model_{country.lower()}.pkl")
    with open(model_filename, "wb") as f:
        pickle.dump(model, f)

    print(f"✅ Modèle entraîné et sauvegardé : {model_filename}")
    print(f"📊 MAE: {mae:.2f} | RMSE: {rmse:.2f}")

if __name__ == "__main__":
    train_model("France") # Entraîner un modèle pour la France
```

- Le modèle de régression linéaire est entraîné sur l'ensemble d'entraînement.
- Le modèle est évalué en utilisant les métriques MAE et RMSE.
- Le modèle entraîné est sauvegardé en utilisant pickle.
- Le script peut être exécuté pour entraîner un modèle spécifique à un pays.
- Les données sont divisées en ensembles d'entraînement et de test avec une répartition 80/20.

9. Conclusion

L'entraînement des données suit une approche basée sur la régression linéaire pour modéliser l'évolution des cas pandémiques en fonction du temps et des facteurs épidémiologiques. Cette méthode permet de :

- **Simplifier l'interprétation des résultats** grâce à un modèle linéaire facilement compréhensible.
- **Prédire l'évolution des cas pandémiques** en fonction des tendances historiques.
- **Éviter le surajustement** en utilisant un modèle léger adapté aux données disponibles.
- **Faciliter l'intégration dans un pipeline de données** où les modèles peuvent être mis à jour régulièrement avec de nouvelles données.

Le choix de la régression linéaire repose sur sa robustesse et sa capacité à fournir des estimations fiables pour des tendances globales, bien qu'il puisse être complété par des modèles plus avancés si nécessaire.

VII) API de consultation des données

L'API REST a été développée avec **FastAPI** afin de fournir un accès structuré aux données épidémiologiques stockées dans **PostgreSQL**. Elle permet d'effectuer des **requêtes dynamiques**, notamment pour **recupérer, ajouter, modifier et supprimer des données (CRUD)**.

1) Architecture de l'API

L'API repose sur une **architecture RESTful** et suit les principes suivants :

- **FastAPI** pour un serveur rapide et performant.
- **Pydantic** pour la validation et la sérialisation des données.
- **Connexion PostgreSQL via Psycopg2** pour interagir avec la base de données.
- **Documentation automatique** grâce à **Swagger UI** intégrée à FastAPI.

L'API expose plusieurs **endpoints** permettant l'interaction avec les données.

2) Connexion à la base de données

L'API doit interagir avec la base **PostgreSQL**, hébergée en local ou sur un serveur distant.

Nous définissons une fonction pour établir une connexion sécurisée :

```
from fastapi import FastAPI, Query, HTTPException, Depends, Body
import psycopg2
import pandas as pd
from typing import Optional
from pydantic import BaseModel

app = FastAPI()

# Connexion à la base de données PostgreSQL
DATABASE_URL = "dbname=pandemics user=postgres password=admin host=localhost port=5432"

def get_db_connection():
    return psycopg2.connect(DATABASE_URL)
```

Explication du code :

- **Stockage de l'URL de connexion** pour éviter les répétitions.
- **Création d'une connexion** PostgreSQL réutilisable via `psycopg2.connect()`.
- **Gestion sécurisée des accès** (possibilité d'utiliser des **variables d'environnement** pour les credentials).

3) Modèle de données (Pydantic)

Pour assurer la **validation des requêtes**, nous utilisons **Pydantic**, qui permet de **définir un modèle de données structuré**

Modèle Utilisateur \ Mise à Jour Utilisateur

```
class User(BaseModel):  
    username: str  
    email: str  
    password: str  
  
class UserUpdate(BaseModel):  
    username: Optional[str] = None  
    email: Optional[str] = None  
    password: Optional[str] = None
```

Modèle Data \ Mise à Jour Data

```
# Modèle Pydantic pour la validation des entrées
class PandemicData(BaseModel):
    user_id: int
    country: str
    date: str
    cases: int
    deaths: int
    recovered: int
    active: int
    latitude: Optional[float] = None
    longitude: Optional[float] = None
    who_region: Optional[str] = None
    mortality_rate: Optional[float] = None
    recovery_rate: Optional[float] = None

class PandemicUpdate(BaseModel):
    cases: Optional[int] = None
    deaths: Optional[int] = None
    recovered: Optional[int] = None
    active: Optional[int] = None
    latitude: Optional[float] = None
    longitude: Optional[float] = None
    who_region: Optional[str] = None
    mortality_rate: Optional[float] = None
    recovery_rate: Optional[float] = None
```

Explication du code :

- Définition d'un **modèle Pydantic** pour valider les entrées utilisateurs.
- Ajout de **champs optionnels** (latitude, longitude, who_region, etc.).
- Sécurisation de la **cohérence des données** en forçant certains types (int, float, str)

4) Endpoints de l'API Pandemics

a) Test de connexion à la base de données PostgreSQL

L'endpoint **POST /test_connection/** permet de **vérifier l'état de la connexion** à la base de données PostgreSQL.

```
# Test de connexion
@app.post("/test_connection/")
def test_connection():
    try:
        conn = get_db_connection()
        conn.close()
        return {"status": "✅ Connexion réussie à PostgreSQL"}
    except Exception as e:
        return {"status": "❌ Échec de connexion", "error": str(e)}
```

Explication du code :

1. Connexion à PostgreSQL

- La fonction `get_db_connection()` est appelée pour tenter d'ouvrir une connexion à la base de données.
- Si la connexion est établie avec succès, elle est immédiatement **fermée** pour éviter une surcharge inutile.

2. Gestion des erreurs

- Si une erreur survient (Exception), elle est capturée et retournée dans la réponse JSON.
- L'erreur est convertie en **chaîne de texte** (`str(e)`) pour fournir un message explicite sur l'échec de connexion.

3. Retour d'une réponse JSON

- **Succès** : Message indiquant que la connexion est établie avec PostgreSQL.
- **Échec** : Message d'erreur détaillant la cause du problème (ex: mauvais identifiants, serveur PostgreSQL arrêté, etc.).
-

b) Récupération des données (GET)

L'endpoint **GET /data/** permet de récupérer les données de la base en appliquant **en utilisant l'utilisateur id**.

Étape 1 : Définition des paramètres de requête

L'API accepte un paramètre `user_id` qui correspond à l'id de l'utilisateur connecté.


```

@app.get("/data/{user_id}")
def get_data(user_id: int):
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT id, country, latitude, longitude, date, cases, deaths, recovered, active, who_region, mortality_rate, recovery_rate
        FROM pandemic_data
        WHERE user_id = %s
    """, (user_id,))

    rows = cursor.fetchall()

    # Vérifier si des données existent pour cet utilisateur
    if not rows:
        raise HTTPException(status_code=404, detail="Aucune donnée trouvée pour cet utilisateur")

    # Récupérer les noms de colonnes
    colnames = [desc[0] for desc in cursor.description]

    cursor.close()
    conn.close()

    # Transformer en JSON propre
    result = [dict(zip(colnames, row)) for row in rows]

    return result # FastAPI convertira automatiquement en JSON

```

Explication du code

- **Définition de l'endpoint** : `@app.get("/data/{user_id}")` → Permet d'obtenir les données pandémiques d'un utilisateur via son `user_id`.
- **Connexion à la base de données** : `get_db_connection()` → Établit la connexion PostgreSQL et crée un curseur pour exécuter des requêtes SQL.
- **Exécution de la requête SQL** :
 - Récupère les données de `pandemic_data` où `user_id` correspond à l'ID fourni.
 - `cursor.fetchall()` stocke les résultats.
- **Gestion des erreurs** :
 - Si aucune donnée n'est trouvée, lève une `HTTPException(404)`.
- **Récupération des noms de colonnes** :
 - `colnames = [desc[0] for desc in cursor.description]` → Associe chaque colonne aux valeurs de la base.
- **Fermeture de la connexion** : `cursor.close()` et `conn.close()` pour libérer les ressources.
- **Transformation des résultats** :
 - Convertit chaque ligne SQL en dictionnaire (`dict(zip(colnames, row))`).
 - Retourne les données sous format JSON.

c) Ajout de nouvelles données (POST)

L'endpoint **POST /data/** permet **d'insérer une nouvelle entrée** dans la base de données. Il garantit également la **mise à jour automatique** si une donnée pour le même pays et la même date existe déjà.

Étape 1 : Connexion à la base de données

Avant d'insérer une donnée, une connexion à PostgreSQL est établie.

```
# Ajouter une nouvelle entrée dans la BDD
@app.post("/data/")
def add_data(entry: PandemicData):
    conn = get_db_connection()
    cursor = conn.cursor()
```

Explication du code :

- `get_db_connection()` est appelé pour **établir une connexion à PostgreSQL**.
- `cursor = conn.cursor()` permet d'exécuter des requêtes SQL.

Étape 2 : Exécution de la requête SQL d'insertion

Une requête `INSERT INTO` est utilisée pour **ajouter une nouvelle entrée**, avec une gestion des **conflits** pour éviter les doublons.

```

cursor.execute("""
    INSERT INTO pandemic_data (country, date, cases, deaths, recovered, active, latitude, longitude,
    who_region, mortality_rate, recovery_rate)
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    ON CONFLICT (country, date)
    DO UPDATE SET
        cases = EXCLUDED.cases,
        deaths = EXCLUDED.deaths,
        recovered = EXCLUDED.recovered,
        active = EXCLUDED.active,
        latitude = EXCLUDED.latitude,
        longitude = EXCLUDED.longitude,
        who_region = EXCLUDED.who_region,
        mortality_rate = EXCLUDED.mortality_rate,
        recovery_rate = EXCLUDED.recovery_rate;
""", (entry.country, entry.date, entry.cases, entry.deaths, entry.recovered, entry.active, entry.latitude,
    entry.longitude, entry.who_region, entry.mortality_rate, entry.recovery_rate))

```

Explication du code :

- INSERT INTO ajoute une nouvelle ligne avec les valeurs fournies.
- ON CONFLICT (country, date) DO UPDATE permet de **mettre à jour automatiquement** une entrée existante avec les nouvelles valeurs envoyées.
- EXCLUDED.<champ> fait référence aux valeurs **proposées par la requête**, remplaçant celles existantes en cas de conflit.

Avantages :

- **Évite les doublons** : Une même entrée (country, date) ne peut pas être insérée deux fois.
- **Met à jour les données existantes automatiquement.**

Étape 3 : Validation et retour de réponse

Une fois la requête exécutée, la connexion est fermée proprement et un message de confirmation est retourné.

```

conn.commit()
cursor.close()
conn.close()
return {"message": "✅ Données insérées avec succès"}

```

Explication du code :

- conn.commit() valide l'opération et **enregistre la transaction** dans la base.

- `cursor.close()` et `conn.close()` ferment proprement la connexion PostgreSQL.
- La réponse JSON confirme que les **données ont bien été insérées ou mises à jour**.

d) Mise à jour des données (PUT)

L'endpoint **PUT /data/update/** permet de **modifier un enregistrement existant** dans la base de données. Il met à jour uniquement les champs **fournis** dans la requête, évitant ainsi d'écraser des données qui ne sont pas spécifiquement modifiées.

Étape 1 : Vérification de l'existence de l'entrée

Avant toute mise à jour, nous devons nous assurer que l'enregistrement ciblé **existe** dans la base de données.

```
# Modifier une entrée existante
@app.put("/data/update/")
def update_data(entry: PandemicData):
    conn = get_db_connection()
    cursor = conn.cursor()

    # Vérifier si l'entrée existe
    cursor.execute("SELECT * FROM pandemic_data WHERE country = %s AND date = %s", (entry.country, entry.date))
    existing_data = cursor.fetchone()

    if not existing_data:
        raise HTTPException(status_code=404, detail="Aucune donnée trouvée pour mise à jour")

    # Construction dynamique de la requête SQL avec uniquement les champs fournis
    update_fields = []
    params = []
```

Explication du code :

- On utilise une requête **SELECT** pour vérifier si un enregistrement **correspondant au pays et à la date** existe.
- Si aucune donnée n'est trouvée, on **renvoie une erreur 404** indiquant que l'entrée n'existe pas

Étape 2 : Construction dynamique de la requête SQL

Nous devons maintenant **préparer dynamiquement** la requête UPDATE, en incluant uniquement les champs **qui ont été spécifiés** dans la requête.

```
if entry.cases is not None:
    update_fields.append("cases = %s")
    params.append(entry.cases)
if entry.deaths is not None:
    update_fields.append("deaths = %s")
    params.append(entry.deaths)
if entry.recovered is not None:
    update_fields.append("recovered = %s")
    params.append(entry.recovered)
if entry.active is not None:
    update_fields.append("active = %s")
    params.append(entry.active)
if entry.latitude is not None:
    update_fields.append("latitude = %s")
    params.append(entry.latitude)
if entry.longitude is not None:
    update_fields.append("longitude = %s")
    params.append(entry.longitude)
if entry.who_region is not None:
    update_fields.append("who_region = %s")
    params.append(entry.who_region)
if entry.mortality_rate is not None:
    update_fields.append("mortality_rate = %s")
    params.append(entry.mortality_rate)
if entry.recovery_rate is not None:
    update_fields.append("recovery_rate = %s")
    params.append(entry.recovery_rate)
```

Explication du code :

- Pour chaque champ **non nul** envoyé dans la requête, on ajoute le champ correspondant dans **update_fields** et sa valeur dans **params**.
- Cela permet d'**éviter de modifier des valeurs qui ne sont pas spécifiées**.

Étape 3 : Exécution de la requête de mise à jour

Une fois les champs construits dynamiquement, nous exécutons la requête SQL UPDATE.

```
if not update_fields:
    raise HTTPException(status_code=400, detail="Aucune donnée à mettre à jour")

# Construire la requête finale
query = f"""
    UPDATE pandemic_data
    SET {', '.join(update_fields)}
    WHERE country = %s AND date = %s
"""
params.append(entry.country)
params.append(entry.date)

cursor.execute(query, tuple(params))
conn.commit()
cursor.close()
conn.close()
return {"message": "✅ Données mises à jour avec succès"}
```

Explication du code :

- Si aucun champ n'a été fourni dans la requête, une **erreur 400 est renvoyée**.
- La requête UPDATE est construite dynamiquement avec les **champs modifiés**.
- On exécute la requête avec `cursor.execute(query, tuple(params))`, puis on **valide la transaction** avec `conn.commit()`.

Cette méthode **évite d'écraser accidentellement des données non spécifiées**.

e) Suppression d'une entrée (DELETE)

L'endpoint **DELETE /data/** permet de **supprimer une entrée spécifique** dans la base de données, identifiée par **son id**.

```

@app.delete("/data/{id}")
def delete_data(id: int):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM pandemic_data WHERE id = %s", (id,))
    if cursor.rowcount == 0:
        raise HTTPException(status_code=404, detail="Donnée non trouvée")
    conn.commit()
    cursor.close()
    conn.close()
    return {"message": "✅ Donnée supprimée avec succès"}

```

Explication du code :

- La requête **DELETE** tente de supprimer une entrée avec l'**id** donnés.
- `cursor.rowcount` est utilisé pour **vérifier si une ligne a été supprimée**.
- Si aucune donnée ne correspond, une **erreur 404** est retournée.
- **Sinon, la suppression est validée avec `commit()`** et un message de confirmation est retourné.

f) Documentation automatique avec Swagger

L'API **génère automatiquement une documentation interactive** accessible via **Swagger UI** à l'adresse : **<http://localhost:8000/docs>**

Permet de tester directement les requêtes.

Affiche les modèles de données et les paramètres attendus.

5) Endpoints de l'API Users

a) Création d'un utilisateur (POST /users/)

Crée un nouvel utilisateur dans la base de données.

```
@app.put("/users/{user_id}")
def update_user(user_id: int, user_update: UserUpdate):
    conn = get_db_connection()
    cursor = conn.cursor()
    update_fields = []
    params = []

    if user_update.username:
        update_fields.append("username = %s")
        params.append(user_update.username)
    if user_update.email:
        update_fields.append("email = %s")
        params.append(user_update.email)
    if user_update.password:
        update_fields.append("password = %s")
        params.append(user_update.password)

    if not update_fields:
        raise HTTPException(status_code=400, detail="Aucune donnée à mettre à jour")

    params.append(user_id)
    query = f"UPDATE users SET {'', '.join(update_fields)} WHERE id = %s"
    cursor.execute(query, tuple(params))
    conn.commit()
    cursor.close()
    conn.close()
    return {"message": "✅ Utilisateur mis à jour avec succès"}
```

b) Récupération d'un utilisateur (GET /users/{user_id})

Récupère les informations d'un utilisateur via son ID.

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, username, email FROM users WHERE id = %s", (user_id,))
    user = cursor.fetchone()
    cursor.close()
    conn.close()
    if not user:
        raise HTTPException(status_code=404, detail="Utilisateur non trouvé")
    return {"id": user[0], "username": user[1], "email": user[2]}
```

c) Mise à jour d'un utilisateur (PUT /users/{user_id})

Modifie les informations d'un utilisateur existant.

```
@app.put("/users/{user_id}")
def update_user(user_id: int, user_update: UserUpdate):
    conn = get_db_connection()
    cursor = conn.cursor()
    update_fields = []
    params = []

    if user_update.username:
        update_fields.append("username = %s")
        params.append(user_update.username)
    if user_update.email:
        update_fields.append("email = %s")
        params.append(user_update.email)
    if user_update.password:
        update_fields.append("password = %s")
        params.append(user_update.password)

    if not update_fields:
        raise HTTPException(status_code=400, detail="Aucune donnée à mettre à jour")

    params.append(user_id)
    query = f"UPDATE users SET {', '.join(update_fields)} WHERE id = %s"
    cursor.execute(query, tuple(params))
    conn.commit()
    cursor.close()
    conn.close()
    return {"message": "✅ Utilisateur mis à jour avec succès"}
```

d) Suppression d'un utilisateur (DELETE /users/{user_id})

Supprime un utilisateur de la base de données.

```
@app.delete("/users/{user_id}")
def delete_user(user_id: int):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM users WHERE id = %s", (user_id,))
    if cursor.rowcount == 0:
        raise HTTPException(status_code=404, detail="Utilisateur non trouvé")
    conn.commit()
    cursor.close()
    conn.close()
    return {"message": "✅ Utilisateur supprimé avec succès"}
```

VIII) Visualisation et exploration des données avec Power BI

1) Objectif de la visualisation des données

L'objectif principal de la visualisation des données est de **fournir une analyse interactive et intuitive** de l'évolution de la pandémie de COVID-19 en exploitant **Power BI**. Grâce aux tableaux de bord créés, les utilisateurs peuvent :

- **Explorer la propagation du virus** par pays et région.
- **Suivre l'évolution des cas, décès et guérisons** au fil du temps.
- **Visualiser la répartition géographique des cas** sur une carte.
- **Comparer les indicateurs de mortalité et de guérison** entre différentes régions du monde.

Power BI a été choisi car il permet :

- Une intégration **directe avec PostgreSQL**.
- La possibilité de **consommer des API REST**.
- Une **interface intuitive et interactive** avec **des filtres dynamiques**.

2) Connexion des données dans Power BI

L'intégration des données dans Power BI repose sur **la connexion via l'api REST**

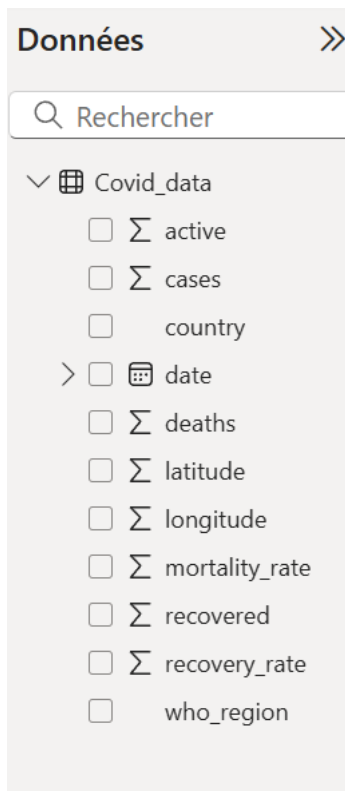


Explication :

- **Utilisation de Web.Contents()** pour récupérer les données via API.
- **Actualisation dynamique** sans dépendance à PostgreSQL.
- Permet **d'interroger les données en temps réel** avec des filtres dynamiques.

3) Structuration des données

Dans Power BI, nous avons importé la table **Covid_data**, qui contient les colonnes suivantes :



Toutes ces données sont utilisées pour créer **des indicateurs visuels pertinents**.

4) Tableaux de bord et visualisations

a) Sélection des filtres interactifs

Power BI permet de **filtrer les données** selon :

- Un pays spécifique.
- Une plage de dates.



The image shows a user interface for filtering data. It consists of two main sections: 'Pays' (Country) and 'Date' (Date). The 'Pays' section has a dropdown menu currently showing 'Tout' (All) with a downward arrow. The 'Date' section has two date pickers: the first shows '22/01/2020' and the second shows '27/07/2020', both with calendar icons to their right.

Exemple de filtre appliqué dans le rapport :

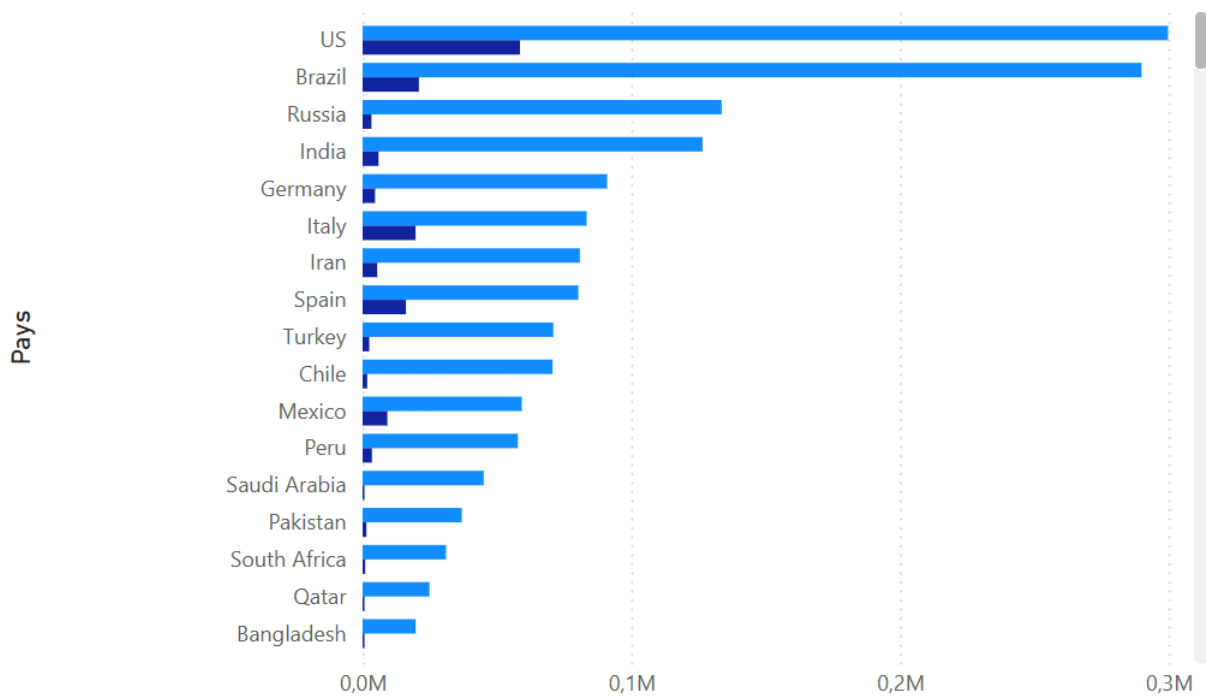
L'utilisateur peut sélectionner **un pays et une période**, puis observer les tendances spécifiques.

b) Moyenne du nombre de décès et de guérisons par pays

Type de visualisation : Graphique en barres horizontales

Objectif : Comparer les **décès et guérisons moyens par pays**.

● Moyenne des guérisons ● Moyenne des décès



Détails :

- L'axe X représente le nombre de cas (moyenne).
- L'axe Y représente les pays.
- Deux couleurs différentes distinguent les décès et guérisons.

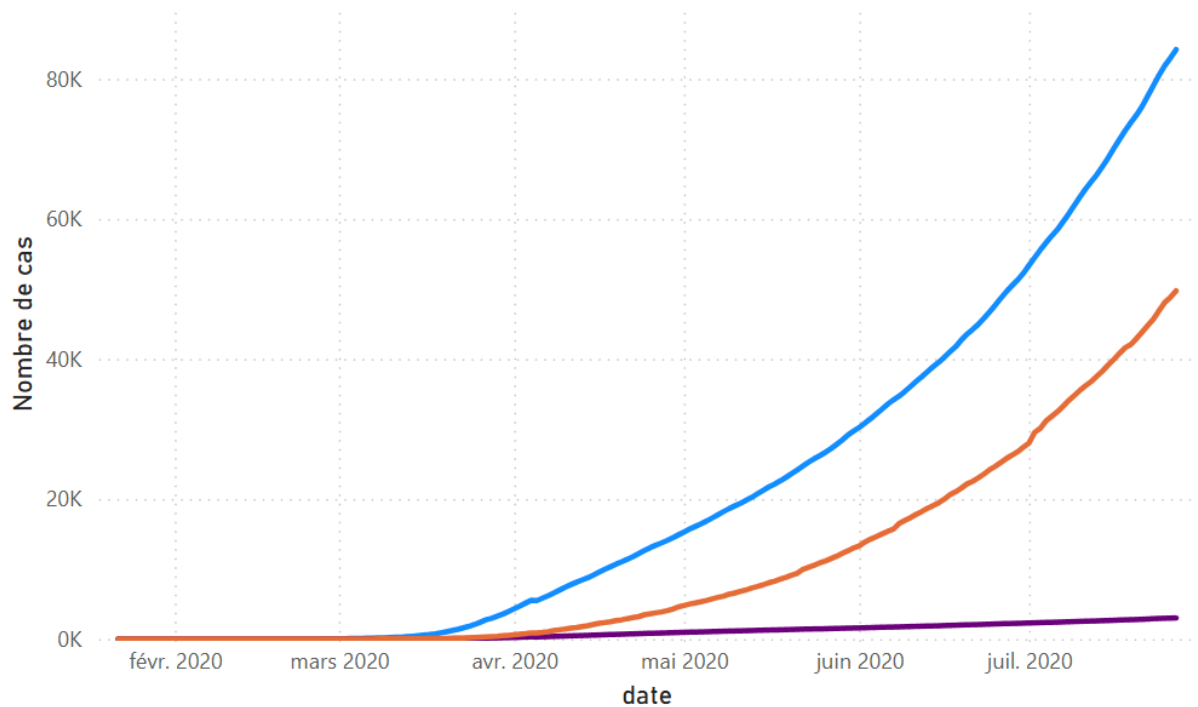
Observations : Les États-Unis et le Brésil affichent le plus grand nombre de décès et guérisons.

c) Courbe d'évolution des cas, décès et guérisons

Type de visualisation : Graphique en courbes

Objectif : Observer la progression des cas au fil du temps.

● Moyenne du nombre de contaminés ● Moyenne du nombre de mort ● Moyenne du nombre de guérissons



Détails :

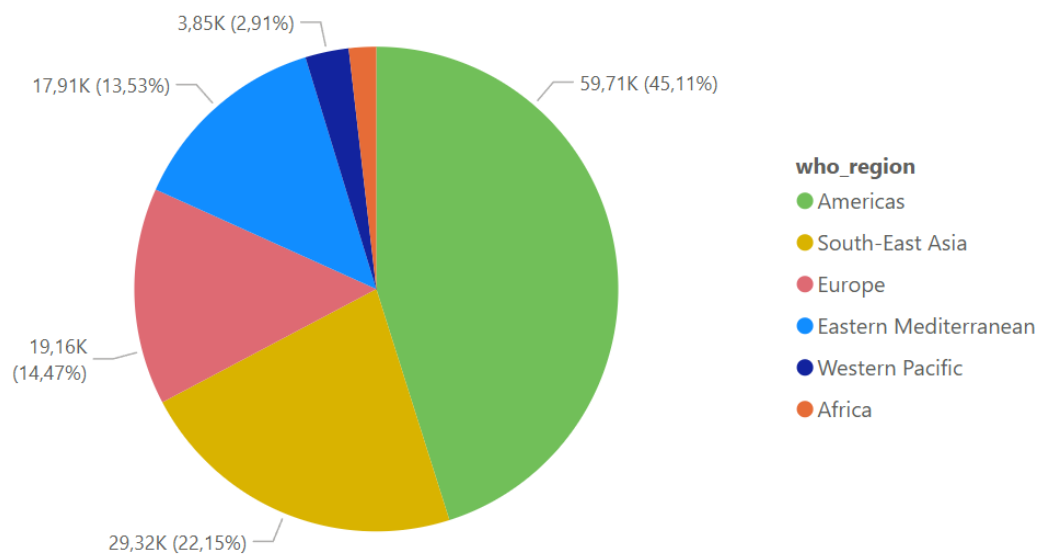
- Trois courbes affichent l'évolution des cas confirmés, décès et guérissons.
- Les valeurs augmentent avec le temps, illustrant la dynamique de propagation du virus.

Observation : La courbe des **cas confirmés** suit une **croissance exponentielle**, avec une **progression plus lente des décès et des guérissons**.

d) Moyenne des cas par région OMS

Type de visualisation : Diagramme circulaire (camembert)

Objectif : Montrer la **répartition des cas** par région définie par l'OMS.



Détails :

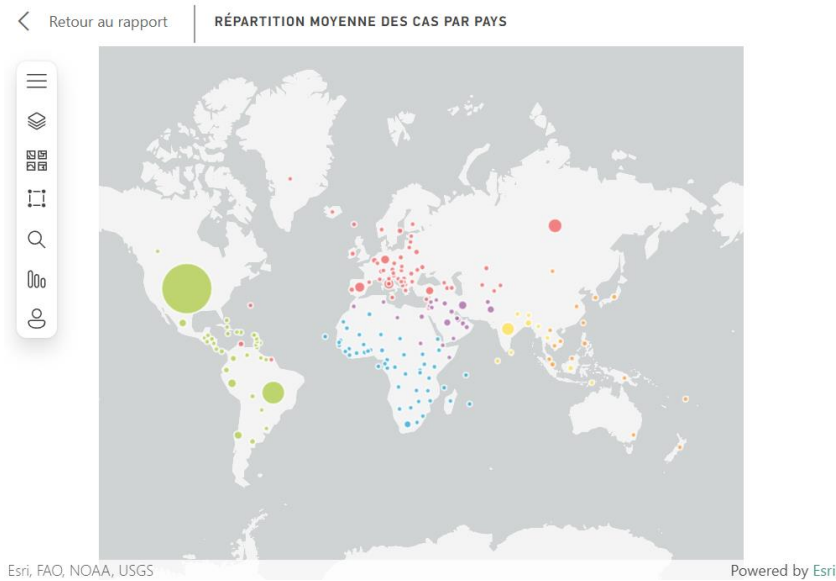
- Les couleurs représentent les différentes régions OMS.
- Les proportions indiquent l'impact relatif du virus selon les zones géographiques.

Observation : L'Amérique est la région la plus touchée, suivie de l'Europe et l'Asie du Sud-Est.

e) Répartition géographique des cas par pays

Type de visualisation : Carte interactive Power BI

Objectif : Représenter visuellement la répartition des cas sur une carte du monde.



Détails :

- Chaque point représente un pays.
- La taille du point est proportionnelle au nombre de cas.
- Code couleur par région OMS pour une lecture plus intuitive.

Observation : Les États-Unis, l'Europe et l'Inde affichent les **plus grandes concentrations de cas**.

f) Indicateurs clés calculés

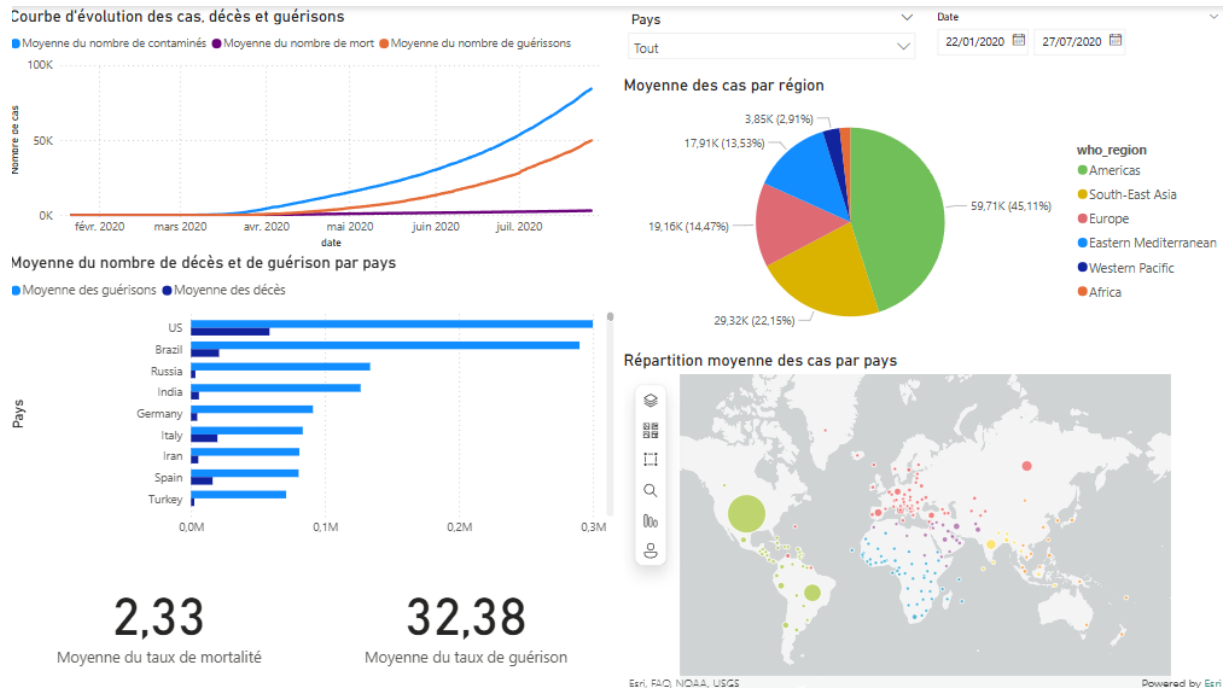
En plus des graphiques, des **indicateurs numériques** sont affichés :

2,33
Moyenne du taux de mortalité

32,38
Moyenne du taux de guérison

Ces indicateurs permettent de comparer rapidement l'impact du virus.

g) Interaction et expérience utilisateur



Pourquoi ces choix de visualisation ?

- **Filtres dynamiques** pour une **expérience interactive**.
- **Comparaisons claires** entre les **pays et régions**.
- **Représentations variées (barres, camemberts, cartes)** pour une **lecture intuitive**.

Avantages de Power BI

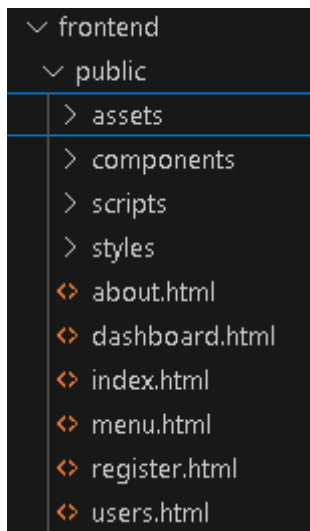
- **Actualisation des données en temps réel** via API REST.
- **Performances optimisées** avec PostgreSQL.
- **Interface visuelle et ergonomique**.

IX) Front-End, Intégration et Gestion des Données

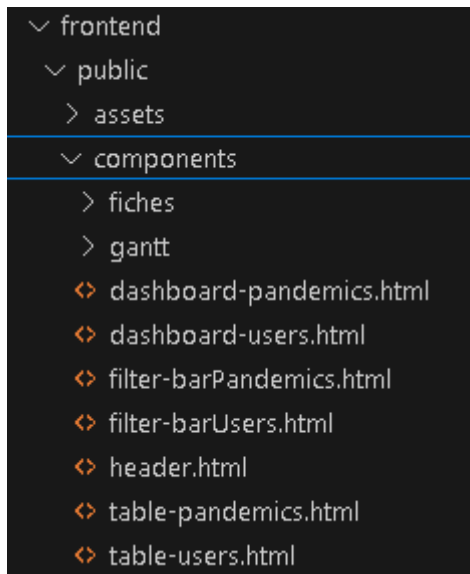
A. Architecture Front-End et Structure des Composants

Pour le front-end, nous avons adopté une approche modulaire et dynamique. La structure du dossier est organisée de la manière suivante :

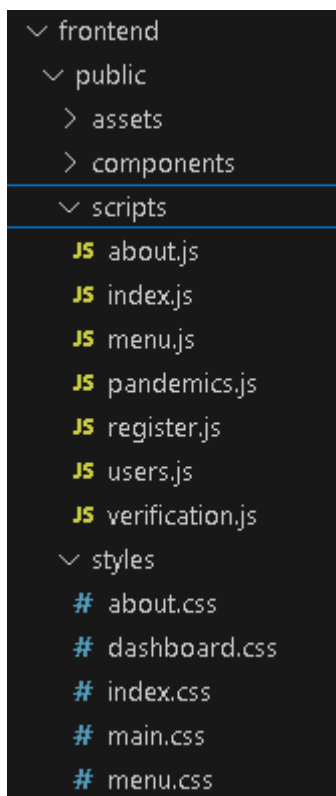
frontend/public : Contient les pages statiques et les ressources (HTML, CSS, JavaScript, images).



frontend/public/components : Regroupe des fragments HTML réutilisables tels que le header, le filter-bar, les dashboards (pandemics et users) et les tableaux.



frontend/public/styles et **frontend/public/scripts** : Respectivement, les feuilles de style et les scripts dédiés à chaque fonctionnalité.



Nous avons utilisé une méthode d'injection dynamique de composants via JavaScript (notamment dans les fichiers comme `pandemics.js` et `users.js`). Cette approche permet de charger les parties communes (header, footer) et spécifiques (dashboard-pandemics, dashboard-users) de manière isolée, facilitant ainsi la maintenance et la réutilisation du code.

```

const componentsUsers = [
  { id: 'header', file: 'components/header.html' },
  { id: 'dashboard-users', file: 'components/dashboard-users.html' },
];

for (const { id, file } of componentsUsers) {
  const response = await fetch(file);
  const html = await response.text();
  document.getElementById(id).innerHTML = html;
}

// Charge les sous-composants du Dashboard
const dashboardComponentsUsers = [
  { id: 'filter-bar', file: 'components/filter-barUsers.html' },
  { id: 'users-table', file: 'components/table-users.html' },
];

for (const { id, file } of dashboardComponentsUsers) {
  const response = await fetch(file);
  const html = await response.text();
  const container = document.querySelector(`.${id}`);

  if (container) {
    container.innerHTML = html;
  } // else {
  //   console.error('🚨 Impossible de trouver l'élément .${id} dans le DOM');
  // }
}

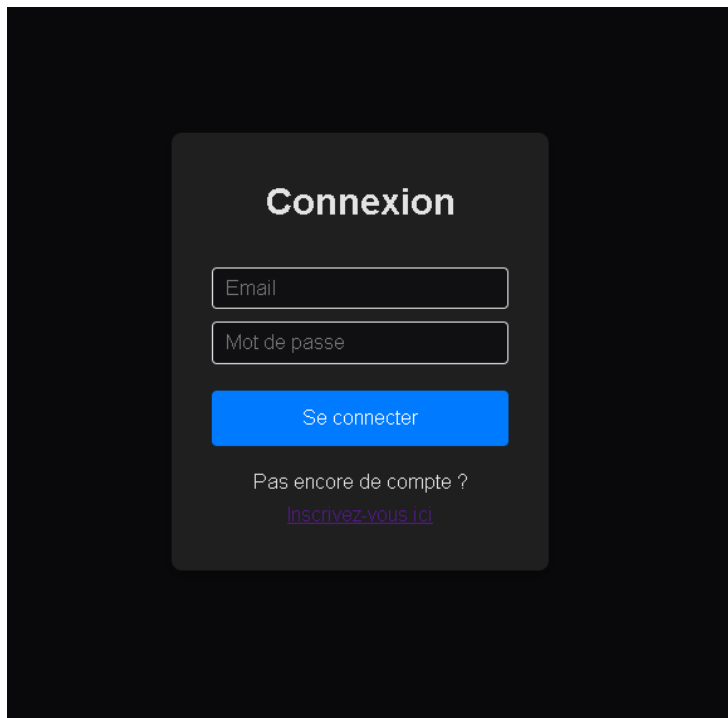
```

B. Design et Expérience Utilisateur

Le design a été pensé pour être moderne et intuitif. Voici quelques points clés de notre démarche :

Interface de Connexion

La page de connexion (fichier index.html) propose un formulaire centré qui, après validation, enregistre un token dans le localStorage et redirige l'utilisateur vers le menu principal.



Exemple de logique : Dès qu'un utilisateur se connecte, un token (ainsi que l'ID et le nom d'utilisateur) est stocké dans le localStorage et/ou dans un cookie pour maintenir la session sur l'ensemble des pages.

```
try {
  const response = await fetch("http://127.0.0.1:5000/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data)
  });
  if (!response.ok) {
    throw new Error("Email ou mot de passe incorrect");
  }
  const result = await response.json();
  console.log("Connexion OK :", result);
  localStorage.setItem("access_token", result.access_token);
  window.location.href = "menu.html";
}
```

Dashboards et Tableaux

Deux dashboards distincts ont été réalisés :

- **Dashboard Pandemics** pour la consultation et la gestion des données pandémiques

Pays : Tous les pays Région OMS : Toutes les régions Date : jj/mm/aaaa

Ajouter une nouvelle donnée

Pays jj/mm/aaaa Cas Décès Rétablis Actifs Latitude
 Longitude Région OMS Taux de mortalité Taux de guérison Ajouter

Pays	Date	Cas	Décès	Guéris	Actifs	Latitude	Longitude	Région OMS	Taux de mortalité	Taux de guérison
China	2020-01-22	10	0	0	10	31.8257	117.2264	Western Pacific	0.000%	0.000%
Japan	2020-01-22	2	0	0	2	36.204824	138.252924	Western Pacific	0.000%	0.000%
South Korea	2020-01-22	1	0	0	1	35.907757000000001	127.766922	Western Pacific	0.000%	0.000%
Taiwan*	2020-01-22	1	0	0	1	23.7	121	Western Pacific	0.000%	0.000%
Thailand	2020-01-22	2	0	0	2	15.870032	100.992541	South-East Asia	0.000%	0.000%
US	2020-01-22	1	0	0	1	40	-100	Americas	0.000%	0.000%
Singapore	2020-01-24	3	0	0	3	1.2833	103.8333	Western Pacific	0.000%	0.000%
Taiwan*	2020-01-24	3	0	0	3	23.7	121	Western Pacific	0.000%	0.000%

○ **Dashboard Users** pour la gestion des utilisateurs

ID : Tous les pays Username : Toutes les régions Date de création : jj/mm/aaaa

Ajouter un Utilisateur

Nom d'utilisateur Email Mot de passe Ajouter

Liste des Utilisateurs

ID	Username	Email	Password	Created_at		
1	default_user	default@email.com	hashed_password	13/03/2025 01:52:16	Modifier	Supprimer
2	string	string	string	13/03/2025 13:36:20	Modifier	Supprimer
3	Brahim	bchaouki00@gmail.com	1234	13/03/2025 01:52:16	Modifier	Supprimer
4	admin	admin@admin	admin	13/03/2025 13:36:20	Modifier	Supprimer
7	test	test@test	test	16/03/2025 19:15:00	Modifier	Supprimer
8	ouie	oui@oui		16/03/2025 20:44:26	Modifier	Supprimer
9	e	e@e	e	16/03/2025 22:14:10	Modifier	Supprimer

Animations et Transitions

Nous avons intégré des animations CSS (effet hover, fade-in, translations) afin d'améliorer l'expérience utilisateur et rendre l'interface plus réactive et moderne.

```

.doc-card-content a {
  display: inline-block;
  padding: 0.4rem 0.8rem;
  background-color: #007bff;
  color: #fff;
  text-decoration: none;
  border-radius: 3px;
  font-size: 0.85rem;
  transition: background-color 0.3s ease;
}

.doc-card-content a:hover {
  background-color: #0056b3;
}

@keyframes fadeInUp {
  0% {
    opacity: 0;
    transform: translateY(20px);
  }
  100% {
    opacity: 1;
    transform: translateY(0);
  }
}

```

Intégration d'images et de liens

Pour enrichir la documentation et le design, nous avons intégré des images et des liens vers divers outils :

Figma : Le prototype de l'interface est disponible sur Figma,
<https://www.figma.com/design/A8Q3R8Ktb1RcXju6yQ1hHS/BAAI?node-id=0-1&t=Va868I6zU9VjiADm-1>.

GitHub : Le dépôt du projet est hébergé sur GitHub. Consultez-le ici :
<https://github.com/M10-white/MSPR-DATA.git>.

Diagramme de Gantt : Un diagramme de Gantt (réalisé avec GanttProject) est intégré dans la documentation pour illustrer la planification du projet.

Cahier des Charges : Des documents PDF décrivant les exigences fonctionnelles et techniques du projet sont référencés dans la fiche technique comme le cahier de charges ou le diagramme UML. (Accessible sur le dépôt GitHub)

D. Authentification et Gestion de Session

L'authentification est basée sur un système de token. Lorsqu'un utilisateur se connecte via le formulaire de connexion, les identifiants sont envoyés à l'endpoint /login de l'API. En cas de succès, la réponse contient notamment un token (ainsi que le user_id et le username) qui est ensuite stocké dans le localStorage.

POST

/ login Login

^

Vérifie si l'email et le mot de passe correspondent à un user en base.

Parameters

Try it out

No parameters

Request body required

application/json

Example Value

Schema

```
{
  "email": "user@example.com",
  "password": "string"
}
```

Ce token est vérifié sur chaque page protégée pour s'assurer que l'utilisateur est connecté. Ainsi, l'accès aux dashboards et autres pages sensibles est restreint.

```
const token = localStorage.getItem("access_token");
const logoutBtn = document.getElementById("logoutBtn");
if (token && logoutBtn) {
  logoutBtn.style.display = "block";
} else if (logoutBtn) {
  logoutBtn.style.display = "none";
}

if (logoutBtn) {
  logoutBtn.addEventListener("click", () => {
    console.log("Bouton de déconnexion cliqué");
    localStorage.removeItem("access_token");
    window.location.href = "index.html";
  });
}

if (!token) {
  window.location.href = "/";
}
```

E. Filtrage des Données

Les tableaux de données (tant pour les pandémies que pour les utilisateurs) disposent d'une barre de filtres permettant de :

Par exemple pour le dashboard "pandemics" :

Sélectionner un **pays** dans un menu déroulant.

Pays : Tous les pays ▼

- Tous les pays
- Singapore
- Japan
- China
- South Korea
- Thailand
- US
- Vietnam
- France
- Emirat Arabe Uni
- Angleterre
- Sri Lanka
- Espagne
- Suède
- Suisse

Choisir une **région OMS** pour les données pandémiques.

Région OMS : Toutes les régions ▼

- Toutes les régions
- Western Pacific
- South-East Asia
- Americas
- Europe

Définir une **plage de dates**.

Date :

mars 2025 ▾

↑ ↓

lu	ma	me	je	ve	sa	di
24	25	26	27	28	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Effacer

Aujourd'hui

Ces filtres sont gérés via des écouteurs d'événements qui déclenchent une fonction de filtrage.

```
function waitForFilterElements() {
  const countrySelect = document.getElementById("country");
  const omsSelect = document.getElementById("omsRegion");
  const dateInput = document.getElementById("date");

  if (!countrySelect || !omsSelect || !dateInput) {
    setTimeout(waitForFilterElements, 500);
    return;
  }

  // Les éléments existent, on attache les écouteurs
  countrySelect.addEventListener("change", applyFilters);
  omsSelect.addEventListener("change", applyFilters);
  dateInput.addEventListener("change", applyFilters);
}

waitForFilterElements();
```

Celle-ci filtre le tableau des données en fonction des critères sélectionnés et réaffiche la page correspondante. Cette fonctionnalité permet une navigation rapide et ciblée dans de grandes quantités de données.

X) Benchmark des solutions

Frontend : HTML, CSS, JavaScript

Avantages :

HTML :

Simple et universel, il constitue la base de toute page web et garantit une compatibilité sur tous les navigateurs.

CSS :

Permet de créer des designs flexibles et responsifs. Il offre la possibilité de réaliser des animations et des transitions pour améliorer l'expérience utilisateur.

JavaScript :

Fournit la dynamique nécessaire pour rendre l'interface interactive. Il est essentiel pour gérer des opérations telles que le chargement dynamique de composants, les interactions utilisateur (CRUD, filtrage, pagination) et la communication avec l'API via AJAX.

Inconvénients :

HTML :

Sa nature statique peut limiter la réactivité sans l'aide de JavaScript.

CSS :

Sans une méthodologie adaptée (comme BEM), la gestion d'un grand projet peut devenir complexe et difficile à maintenir.

JavaScript :

Bien que puissant, il peut parfois poser des problèmes de performance ou de compatibilité entre différents navigateurs, surtout dans des applications très complexes.

Backend et APIs : FastAPI, Flask et Uvicorn

Avantages :

FastAPI :

C'est un framework très rapide qui offre une validation automatique des données via Pydantic, ce qui est idéal pour développer des APIs modernes et robustes.

Flask :

Léger et flexible, il convient parfaitement aux projets de plus petite envergure ou aux prototypes rapides.

Uvicorn :

Ce serveur ASGI est performant et est spécialement optimisé pour fonctionner avec FastAPI, garantissant ainsi une exécution efficace des requêtes.

Inconvénients :

FastAPI :

Sa courbe d'apprentissage peut être plus élevée pour les développeurs habitués à des frameworks plus simples comme Flask.

Flask :

Il peut montrer ses limites en termes de performance et de scalabilité pour les applications de grande envergure.

Uvicorn :

Pour en tirer pleinement parti, il doit être associé à un framework ASGI ; isolément, ses fonctionnalités sont moins exploitables.

Bases de Données : PostgreSQL, SQLAlchemy et Psycopg

Avantages :

PostgreSQL :

Une base robuste qui supporte les transactions ACID et les requêtes complexes.

SQLAlchemy :

Cet ORM facilite la gestion des bases relationnelles en fournissant une interface de haut niveau pour interagir avec la base de données.

Psycopg :

C'est une bibliothèque optimisée pour PostgreSQL, reconnue pour sa rapidité et sa stabilité.

Inconvénients :

PostgreSQL :

Sur de très gros volumes de données, certaines requêtes peuvent être plus lentes comparées à des solutions NoSQL.

SQLAlchemy :

Sa syntaxe et sa courbe d'apprentissage peuvent être déroutantes pour les débutants.

Psycopg :

La configuration peut parfois s'avérer délicate, notamment en ce qui concerne la gestion des connexions et des pools.

Traitement des Données : Pandas, Pydantic et Email-validator

Avantages :

Pandas :

Très efficace pour la manipulation de données tabulaires, il permet de réaliser des opérations complexes en quelques lignes de code.

Pydantic :

Offre une validation stricte et sécurisée des données, ce qui aide à prévenir les erreurs en amont dans le traitement des données.

Email-validator :

Permet de vérifier la validité des adresses email et d'éviter les erreurs de saisie dans les formulaires.

Inconvénients :

Pandas :

Peut consommer beaucoup de mémoire lorsque l'on traite des volumes importants de données.

Pydantic :

Bien que très utile, sa validation peut être plus lente que des solutions plus simples lorsque le volume de données est très important.

Email-validator :

Sa fonctionnalité se limite strictement à la validation d'emails et ne couvre pas d'autres types de vérifications.

Automatisation et Orchestration : Prefect et Prefect-SQLAlchemy

Avantages :

Prefect :

Permet la gestion des workflows et l'automatisation des tâches complexes, ce qui est essentiel pour orchestrer un pipeline ETL.

Prefect-SQLAlchemy :

Facilite l'intégration avec SQLAlchemy pour la gestion des interactions avec la base de données, améliorant ainsi l'automatisation des tâches liées aux données.

Inconvénients :

Prefect :

La configuration initiale peut être complexe et la documentation, bien que complète, nécessite une lecture attentive pour une utilisation optimale.

Prefect-SQLAlchemy :

Il est moins répandu que d'autres solutions d'automatisation comme Celery, ce qui peut limiter le support communautaire.

Méthodologie Agile et Gestion de Projet

Pourquoi la méthodologie Agile et une gestion de projet structurée ?

Itérations courtes et flexibilité :

Permet de livrer continuellement des fonctionnalités, de s'adapter aux retours et de réagir rapidement aux changements de priorités.

Collaboration renforcée :

Les réunions quotidiennes et les rétrospectives favorisent une communication efficace entre les membres de l'équipe.

Traçabilité et qualité :

Grâce à l'utilisation de GitHub avec des branches protégées, des merge requests et des commits conventionnels, nous assurons une intégration continue et une qualité de code constante.

Utilisation dans le projet :

Le développement est organisé en sprints, chacun avec des objectifs définis et des revues régulières pour ajuster la trajectoire.

Le dépôt GitHub sert de référentiel central où chaque fonctionnalité est développée sur une branche dédiée avant d'être intégrée via des merge requests.

Les commits suivent un format standard (conventional commits) pour une meilleure lisibilité et traçabilité de l'historique des modifications.